

TP d’attaque par extension de longueur de hachage

SEED Labs, 2019

1 Introduction

Lorsqu’un client et un serveur communiquent sur Internet, ils sont exposés aux attaques MITM. Un attaquant peut intercepter la requête du client. Il peut choisir de modifier les données et d’envoyer la requête modifiée au serveur. Dans ce cas, le serveur doit vérifier l’intégrité de la requête reçue. La méthode standard pour vérifier l’intégrité de la requête consiste à lui associer une étiquette appelée MAC. Il existe plusieurs façons de calculer un MAC, et certaines de ces méthodes ne sont pas sécurisées.

Le MAC est calculé à partir d’une clé secrète et d’un message. Une méthode naïve pour calculer un MAC consiste à concaténer la clé avec le message et à calculer le hachage à sens unique de la chaîne obtenue. Cette méthode semble correcte, mais elle est vulnérable à une attaque appelée attaque par extension de longueur, qui permet aux attaquants de modifier le message tout en étant capables de générer un MAC valide basé sur le message modifié, sans connaître la clé secrète.

L’objectif de ce TP est d’aider les étudiants à comprendre le fonctionnement de l’attaque par extension de longueur. Les étudiants lanceront cette attaque contre un programme serveur ; ils forgeront une commande valide et amèneront le serveur à exécuter cette commande.

Lectures. Une présentation détaillée des fonctions de hachage à sens unique peut être trouvée dans la ressource suivante :

— Chapitre 22 du livre SEED, *SEED Labs Book*

Environnement du TP. L’environnement du TP est fourni par les fichiers de configuration du laboratoire SEED.

2 Environnement du TP

Nous avons configuré un serveur web pour ce TP. Un client peut envoyer une liste de commandes à ce serveur. Chaque requête doit contenir un MAC calculé à partir d’une clé secrète et de la liste des commandes. Le serveur n’exécutera les commandes de la requête que si le MAC est vérifié avec succès. Nous utiliserons la VM hôte comme client et un conteneur pour le serveur web.

Configuration et commandes du conteneur. Les commandes de configuration du conteneur dépendent de l’environnement Docker fourni avec le TP.

Pour démarrer l’environnement, utilisez les fichiers et commandes fournis dans le répertoire `Labsetup`.

À propos du serveur web. Nous utilisons le domaine `www.seedlab-hashlen.com` pour héberger le programme serveur. Dans notre VM, nous faisons correspondre ce nom d’hôte au conteneur du serveur web (10.9.0.80). Cela peut être réalisé en ajoutant l’entrée suivante au fichier `/etc/hosts` (veuillez l’ajouter si cette entrée n’est pas présente dans la VM).

```
10.9.0.80 www.seedlab-hashlen.com
```

Le code du serveur se trouve dans le dossier `Labsetup/image_flask/app`. Il contient deux répertoires. Le répertoire `www` contient le code du serveur, tandis que le répertoire `LabHome` contient un fichier secret et la clé utilisée pour calculer le MAC.

Envoi de requêtes. Le programme serveur accepte les commandes suivantes :

- La commande `lstcmd` : le serveur liste tous les fichiers du dossier `LabHome`.
- La commande `download` : le serveur renvoie le contenu du fichier spécifié dans le répertoire `LabHome`.

Un exemple de requête envoyée par le client au serveur est présenté ci-dessous. Le serveur exige qu'un argument `uid` soit fourni. Il utilise `uid` pour obtenir la clé MAC depuis `LabHome/key.txt`. La commande dans l'exemple ci-dessous est `lstcmd`, et sa valeur est fixée à 1. Elle demande au serveur de lister tous les fichiers. Le dernier argument est le MAC calculé à partir de la clé secrète (partagée entre le client et le serveur) et des arguments de la commande. Avant d'exécuter la commande, le serveur vérifie le MAC afin de garantir l'intégrité de la commande.

```
http://www.seedlab-hashlen.com/?myname=JohnDoe&uid=1001&lstcmd=1
&mac=dc8788905dbcbceffcd5578887717c12691b3cf1dac6b2f2bcfabcd14a6a7f11
```

Les étudiants doivent remplacer la valeur `JohnDoe` dans le champ `myname` par leur vrai nom (aucun espace n'est autorisé). Ce paramètre permet de s'assurer que les résultats des différents étudiants sont différents, afin qu'ils ne puissent pas copier les uns sur les autres. Le serveur n'utilise pas cet argument, mais vérifie s'il est présent ou non. Les requêtes seront rejetées si ce champ n'est pas inclus. Les enseignants peuvent utiliser cet argument pour vérifier si les étudiants ont réalisé le travail eux-mêmes. Aucun point ne sera attribué si les étudiants n'utilisent pas leur vrai nom dans cette tâche.

L'exemple suivant montre une autre requête. Elle contient deux commandes : lister tous les fichiers et télécharger le fichier `secret.txt`. De même, un MAC valide doit être associé à la requête, sinon le serveur n'exécutera pas ces commandes.

```
http://www.seedlab-hashlen.com/?myname=JohnDoe&uid=1001&lstcmd=1
&download=secret.txt
&mac=dc8788905dbcbceffcd5578887717c12691b3cf1dac6b2f2bcfabcd14a6a7f11
```

3 Tâches

3.1 Tâche 1 : Envoyer une requête pour lister les fichiers

Dans cette tâche, nous allons envoyer une requête légitime au serveur afin de voir comment celui-ci répond à la requête. La requête que nous souhaitons envoyer est la suivante :

```
http://www.seedlab-hashlen.com/?myname=<name>&uid=<need-to-fill>
&lstcmd=1&mac=<need-to-calculate>
```

Pour envoyer une telle requête, en plus d'utiliser nos vrais noms, nous devons renseigner les deux arguments manquants. Les étudiants doivent choisir un numéro `uid` dans le fichier `key.txt` du répertoire `LabHome`. Ce fichier contient une liste de valeurs `uid` et clé séparées par des deux-points. Les étudiants peuvent utiliser n'importe quel `uid` et la clé qui lui est associée. Par exemple, ils peuvent utiliser l'`uid` 1001 et sa clé 123456.

Le deuxième argument manquant est le MAC, qui peut être calculé en concaténant la clé avec le contenu de la requête R (uniquement la partie correspondant aux arguments), avec deux-points entre les deux. Voir l'exemple suivant :

```
Key:R = 123456:myname=JohnDoe&uid=1001&lstcmd=1
```

Le MAC sera calculé à l'aide de la commande suivante :

```
$ echo -n "123456:myname=JohnDoe&uid=1001&lstcmd=1" | sha256sum
7d5f750f8b3203bd963d75217c980d139df5d0e50d19d6dfdb8a7de1f8520ce3 -
```

Nous pouvons ensuite construire la requête complète et l'envoyer au programme serveur à l'aide du navigateur :

```
http://www.seedlab-hashlen.com/?myname=JohnDoe&uid=1001&lstcmd=1
&mac=7d5f750f8b3203bd963d75217c980d139df5d0e50d19d6dfdb8a7de1f8520ce3
```

Tâche. Envoyez une commande de téléchargement au serveur et montrez que vous pouvez récupérer le résultat.

3.2 Tâche 2 : Créer le bourrage (padding)

Pour réaliser l'attaque par extension de longueur du hachage, nous devons comprendre comment le bourrage est calculé pour un hachage à sens unique. La taille de bloc de SHA-256 est de 64 octets, donc un message M sera complété jusqu'à un multiple de 64 octets lors du calcul du hachage. Selon la RFC 6234, le bourrage de SHA256 est constitué d'un octet `\x80`, suivi de plusieurs zéros, puis d'un champ de longueur de 64 bits (8 octets) (la longueur correspond au nombre de **bits** du message M).

Supposons que le message original soit `M = "This is a test message"`. La longueur de M est de 22 octets, donc le bourrage est de $64 - 22 = 42$ octets, dont 8 octets pour le champ de longueur. La longueur de M en bits est $22 * 8 = 176 = 0xB0$. SHA256 sera calculé sur le message complété suivant :

```
"This is a test message"
"\x80"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
"\x00\x00\x00"
"\x00\x00\x00\x00\x00\x00\x00\xB0"
```

Il convient de noter que le champ de longueur utilise l'ordre des octets Big-Endian, c'est-à-dire que si la longueur du message est `0x012345`, le champ de longueur dans le bourrage doit être :

```
"\x00\x00\x00\x00\x00\x01\x23\x45"
```

Tâche. Les étudiants doivent construire le bourrage pour le message suivant (les valeurs réelles de `<key>` et `<uid>` doivent être obtenues à partir du fichier `LabHome/key.txt`).

```
<key>:myname=<name>&uid=<uid>&lstcmd=1
```

Il convient de noter que dans l'URL, tous les nombres hexadécimaux du bourrage doivent être encodés en remplaçant `\x` par `%`. Par exemple, `\x80` dans le bourrage doit être remplacé par `%80` dans l'URL ci-dessus. Côté serveur, les données encodées dans l'URL seront reconverties en valeurs binaires. Voir l'exemple suivant :

```
"\x80\x00\x00\x99" should be encoded as "%80%00%00%99"
```

3.3 Tâche 3 : L'attaque par extension de longueur

Dans cette tâche, nous allons générer un MAC valide pour une URL sans connaître la clé MAC. Supposons que nous connaissions le MAC d'une requête valide R, ainsi que la taille de la clé MAC. Notre objectif est de forger une nouvelle requête à partir de R, tout en étant encore capables de calculer un MAC valide.

À partir du message original `M="This is a test message"` et de sa valeur MAC, nous allons montrer comment ajouter un message `"Extra message"` à la fin du M complété, puis calculer son MAC sans connaître la clé MAC secrète.

```
$ echo -n "This is a test message" | sha256sum
6f3438001129a90c5b1637928bf38bf26e39e57c6e9511005682048bedbef906
```

Le programme ci-dessous peut être utilisé pour calculer le MAC du nouveau message :

```
/* length_ext.c */
#include <stdio.h>
#include <arpa/inet.h>
#include <openssl/sha.h>

int main(int argc, const char *argv[])
{
    int i;
    unsigned char buffer[SHA256_DIGEST_LENGTH];
    SHA256_CTX c;

    SHA256_Init(&c);
    for(i=0; i<64; i++)
        SHA256_Update(&c, "*", 1);

    // MAC du message original M (avec bourrage)
    c.h[0] = htobe32(0x6f343800);
    c.h[1] = htobe32(0x1129a90c);
    c.h[2] = htobe32(0x5b163792);
    c.h[3] = htobe32(0x8bf38bf2);
    c.h[4] = htobe32(0x6e39e57c);
    c.h[5] = htobe32(0x6e951100);
    c.h[6] = htobe32(0x5682048b);
    c.h[7] = htobe32(0xedbef906);

    // Ajouter le message supplémentaire
    SHA256_Update(&c, "Extra message", 13);
    SHA256_Final(buffer, &c);

    for(i = 0; i < 32; i++) {
        printf("%02x", buffer[i]);
    }
    printf("\n");
    return 0;
}
```

Les étudiants peuvent compiler le programme comme suit :

```
$ gcc length_ext.c -o length_ext -lcrypto
```

Tâche. Les étudiants doivent d'abord générer un MAC valide pour la requête suivante (<uid> et la clé MAC doivent être obtenus à partir du fichier LabHome/key.txt) :

```
http://www.seedlab-hashlen.com/?myname=<name>&uid=<uid>
&lstcmd=1&mac=<mac>
```

À partir de la valeur <mac> calculée ci-dessus, construisez une nouvelle requête qui inclut la commande `download`. Vous n'êtes pas autorisés à utiliser la clé secrète cette fois-ci. L'URL se présente comme suit.

```
http://www.seedlab-hashlen.com/?myname=<name>&uid=<uid>
&lstcmd=1<padding>&download=secret.txt&mac=<new-mac>
```

Envoyez la requête construite au serveur et montrez que vous pouvez récupérer avec succès le contenu du fichier `secret.txt`.

3.4 Tâche 4 : Atténuation de l'attaque à l'aide de HMAC

Dans les tâches précédentes, nous avons observé les dommages causés lorsqu'un développeur calcule un MAC de manière non sécurisée en concaténant la clé et le message. Dans cette tâche, nous allons corriger l'erreur commise par le développeur. La méthode standard pour calculer des MAC est d'utiliser HMAC. Les étudiants doivent modifier la fonction `verify_mac()` du programme serveur et utiliser le module Python `hmac` pour calculer le MAC. La fonction se trouve dans `lab.py`. Étant donné une clé et un message (tous deux de type chaîne), le HMAC peut être calculé comme indiqué ci-dessous (si vous copiez-collez le code depuis ce fichier PDF, les caractères `'` peuvent ne pas être copiés correctement sur certaines plateformes).

```
real_mac = hmac.new(bytearray(key.encode('utf-8')),
                      msg=message.encode('utf-8', 'surrogateescape'),
                      digestmod=hashlib.sha256).hexdigest()
```

Après avoir effectué les modifications, arrêtez tous les conteneurs, reconstruisez-les, puis redémarrez-les tous. Les modifications prendront alors effet. Les étudiants doivent répéter la tâche 1 pour envoyer une requête de liste des fichiers en utilisant HMAC pour le calcul du MAC. En supposant que la clé choisie est 123456, le HMAC peut être calculé avec le programme suivant.

```
#!/bin/env python3

import hmac
import hashlib

key='123456'
message='lstcmd=1'
mac = hmac.new(bytearray(key.encode('utf-8')),
                msg=message.encode('utf-8', 'surrogateescape'),
                digestmod=hashlib.sha256).hexdigest()

print(mac)
```

Les étudiants doivent expliquer pourquoi une requête malveillante utilisant l'extension de longueur et des commandes supplémentaires échouera lors de la vérification du MAC lorsque le client et le serveur utilisent HMAC.

4 Rendu

Le rendu doit contenir les réponses aux tâches demandées, les commandes utilisées et les résultats obtenus.

Présentez clairement les valeurs calculées et les captures ou sorties pertinentes.